

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР2: Реализация микросервисов

Срок: 15.04.2026

Задание:

Опираясь на документ, сформированный в рамках ДЗ4, реализуйте разделение вашего бэкенд-приложения на микросервисы.

Необходимо сделать отчёт по [шаблону](#)

Ход работы

Система разделена на три функциональных микросервиса:

1. **Auth Service (порт 4001):** Ответственен за управление пользователями и безопасность.
2. **Catalog Service (порт 4002):** Управляет данными о ресторанах и их доступности.
3. **Booking Service (порт 4003):** Оркестратор процесса бронирования, координирующий данные из других сервисов.

Паттерн «Database-per-service»

Реализована полная изоляция данных. Вместо одной общей БД созданы три независимых базы данных PostgreSQL: auth_db, catalog_db, booking_db.

- **Изоляция связей:** В коде сущностей (Entities) удалены прямые связи @ManyToOne. Вместо объектов-сущностей используются числовые идентификаторы (userId, restaurantId). Это исключает неявные JOIN-запросы между таблицами разных сервисов.

3. Межсервисное взаимодействие

Взаимодействие между сервисами реализовано по **синхронному протоколу HTTP/REST** с использованием библиотеки axios.

Сценарий «Создание бронирования»:

1. **Клиент** отправляет запрос в Booking Service.

2. **Валидация пользователя:** Booking Service выполняет внутренний POST-запрос к Auth Service (/internal/validate), передавая JWT-токен.
3. **Валидация ресурса:** Booking Service выполняет внутренний GET-запрос к Catalog Service (/internal/restaurants/:id/check), чтобы проверить существование и статус ресторана.
4. **Завершение транзакции:** Только при получении успешных ответов (200 OK) от обоих сервисов, Booking Service сохраняет запись в свою БД.

4. Обработка ошибок и отказоустойчивость

В системе реализована стратегия обработки исключений для предотвращения каскадных сбоев:

- **401 Unauthorized:** Если Auth Service сообщает, что токен невалиден, Booking Service немедленно прекращает операцию и возвращает ошибку клиенту.
- **404 Not Found:** Если Catalog Service не находит ресторан, транзакция бронирования отменяется.
- **500/503 (Fallback):** Все межсервисные вызовы обернуты в блоки try-catch. В случае падения одного из зависимых сервисов, вызывающий сервис не «зависает», а возвращает контролируемую ошибку. Это реализует базовый принцип **Circuit Breaker** (предохранителя).

Shared Types: Вынос общих интерфейсов в отдельную директорию shared для обеспечения консистентности данных между сервисами.

Вывод

В результате данной работы я разделил монолит из прошлой лабораторной на микросервисы